

Course: Advanced Algorithm and C++

Week:01

Department: Computer Science

We live under the shared quote

“ Batch 3 - 3 Departments ”

Supported by:



Organized by:





What We Will Cover in 5 Weeks

Foundation and Memory Management

W1

Sorting Algorithms

W2

Linked List

W3

Stack and Queue

W4

Project

W5



TABLE OF CONTENTS

Reviewing C Foundation	01
From C to C++	02
Stack and Heap Memory	03
Dynamic Memory	04
Pointer	05
Struct	07
Object-Oriented Programming	09



Learning Objectives

By the end of this session, you will be able to:

- **Distinguish** stack from heap memory and choose the right one for a given variable
- **Allocate** and **release** heap memory correctly using **new** and **delete**
- **Use** pointers to read and modify memory, and identify the four common pointer faults
- **Define** a struct and class with data members, member functions, a constructor and a destructor

Reviewing C Foundation

Data Types: int, float, double, char, void

Control Flow: if/else, switch, for, while, do while, break, continue

Functions: Declaration, definition, return type, parameters

Arrays & Strings: Fixed-Sized arrays, char[] strings, null terminator '\0'

```
#include <stdio.h>

int add(int, int); // Function Declaration

int main(){
    char str1[] = "Hello"; // Method 1: String literal
    char str2[] = {'W', 'o', 'r', 'l', 'd', '\0'}; // Method 2: Character array
    int a = 2;
    printf("%s %s\n", str1, str2);
    printf("%d + %d = %d\n", a, 3, add(a, 3)); // Function Call
    if(a > 0){ // Control Flow Statement
        printf("%d is positive\n", a);
    } else {
        printf("%d is not positive\n", a);
    }
    return 0;
}

int add(int a, int b){ // Function Definition
    return a + b;
}
```


From C to C++

C is subset of C++, So most of C syntax can be written in C++

<iostream>: input/output library

cout: Output to console

cin: Read input from console

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;
    cout << "You are " << age << " years old." << endl;
    return 0;
}
```

! You can chain output, just separate them by <<.

Stack and Heap Memory

Choosing the right type of memory region determines the performance and program safety.

	Stack	Heap
Allocation	Automatic	Manual
Speed & Size	Very Fast, Limited Size	Slower, Large(Limited By RAM)
Lifetime	Until out of scope	Until manually freed
Risk	Stack Overflow	Memory Leak

Dynamic Memory

The stack decides for you but the heap does not. You ask for memory and you are responsible for giving it back

1. A single value

```
int *p = new int; // ask for one int on the heap
*p = 26;           // use it
delete p;          // give it back
p = nullptr;       // and stop pointing at it
```

2. An array

```
int *arr = new int[5]; // five ints, side by side
arr[0] = 5;
delete[] arr;
arr = nullptr;
```

The rule that will save you:

- Every new has exactly one matching delete
- new[] released with delete[], never plain delete
- After deleting, set the pointer to nullptr immediately
- Deleting does not destroy the pointer, only the memory it pointed to
- If you lose the pointer before deleting, that memory is gone until the program ends

Pointer

Pointer is a powerful tool in C/C++. It allows us to have direct access to memory address, allowing more dynamic and flexibility.

1. Assigning pointer with Address-of(&)

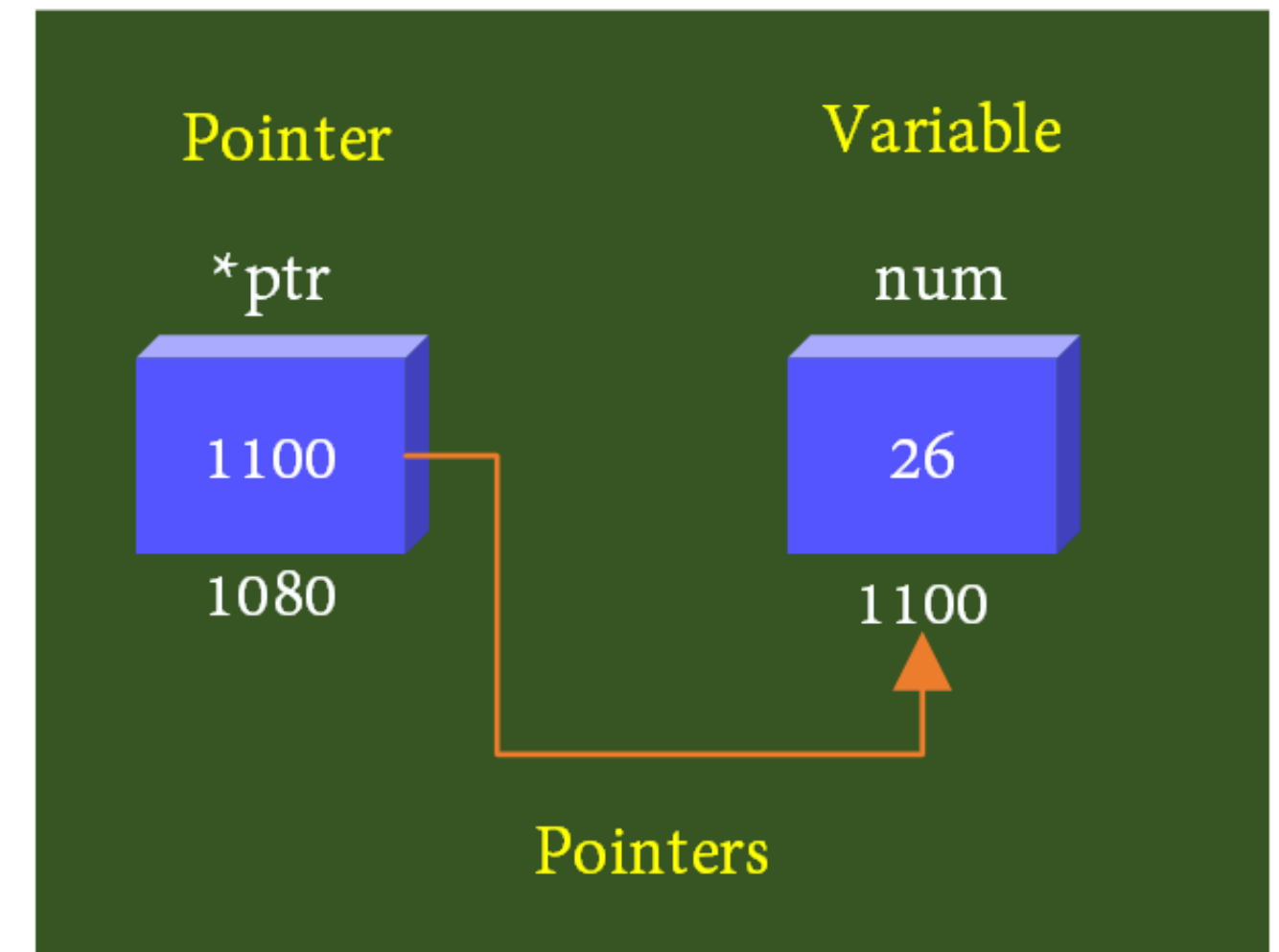
```
int x = 26;  
int *ptr = &x; // ptr hold address of x
```

2. Dereferencing

```
std::cout << *ptr << std::endl; // Read x  
*ptr = 10; // Modify x
```

3. Point at allocated memory

```
int *ptr = new int[2]; // allocate memory  
ptr[0] = 5;           // assign value to first element  
delete[] ptr;         // free memory
```



! A Pointer must have the same type as the data it pointing to.

Pointer

Pointer misuse is the leading cause of crashes:

Pitfall	Description	Prevention
Dangling Pointer	A pointer pointing to a memory location that has already been freed	Set pointer to nullptr after freeing
Wild Pointer	Uninitialized pointer used	Initialize it with nullptr
Memory leak	Memory that is allocated but never freed, while the pointer to its address has lost	Ensure every allocation has a match with a free the memory once the operation are completed
Double free	Freed a memory twice	Set the pointer to nullptr immediately after freeing

```
int *ptr = new int;
*ptr = 10;
delete ptr;
int x = *ptr;    // Dangling pointer

int *ptr2;
*ptr2 = 20;      // Wild pointer

int *ptr3 = new int[5];
ptr3 = new int[5];    // Memory leak

int *ptr4 = new int[5];
delete[] ptr4;
delete[] ptr4;    // Double free
```

! With great pointer, come with great responsibility.

Struct

A struct groups variables of different types under one name so related data travels together

```
#include <iostream>
using namespace std;

struct Student {
    string name;
    int age;
    double gpa;
};

int main() {
    Student s1;           // create
    s1.name = "Dara";      // fill it in
    s1.age = 20;

    Student s2 = {"Sokha", 21, 3.5}; // create and fill at once
    Student classA[3];           // an array of them

    cout << s1.name << " " << s1.age;
}
```

Struct usage:

- Grouping related data under one name
- Creating simple records
- Returning several values from one function
- Passing a set of related values as one argument
- Plain data with no rules to enforce

Struct

A **struct pointer** is a pointer variable that point to a struct.

Two ways to access members via a struct pointer:

- Arrow operator(->)
- Dereferencing the pointer

```
struct Student {  
    std::string name;  
    int age;  
};  
  
int main(){  
    Student student1 = {"John", 20};  
    Student *ptr = &student1; // pointer to struct  
    ptr->name = "Alice"; // access struct members using arrow operator  
    (*ptr).age = 25; // access struct members using dereference operator  
    return 0;  
}
```

! A struct can contain its own pointer but never a copy of itself.

Object-Oriented Programming

Struct store data in one place but functions that act on it somewhere else. **OOP** puts everything together

What we've been doing

```
struct Student {string name;};

void setName(Student &s , string n){
    s.name = n;
}
```

Now using OOP

```
class Student{
    string name;
public:
    void setName(string n){
        name = n;
    };
};
```

Why it's worth the extra syntax

- Data and functions that operate on it are kept together in one place
- **Access control** prevents outside code from directly modifying important data
- **Constructors** and **Destructors** handle **Object's** setup and cleanup automatically

Object-Oriented Programming

Class is also a struct but its members are private unless you say otherwise

struct

```
struct Student{  
    string name; // public  
    int age;     // public  
};
```



class

```
class Student{  
    string name; // private  
    int age;     // private  
};
```

Intended Use:

The difference comes down to their intended use. Use a **struct** when the type mainly holds data that should or can be accessed directly. Use a **class** when the type also control its data, provide behavior or needs to enforce rules.

Object-Oriented Programming

Class holds data and the operations that belong to that data in one place

```
#include <iostream>
using namespace std;

class Student{
private:
    string name; // private
    int age;      // private

public:
    void setName(string n){
        name = n;
    }
    string getName(){
        return name;
    }
};

int main(){
    Student s;
    s.setName("David");
    cout<<s.getName();
}
```

Access specifiers

private - reachable only from inside the class

public - reachable from anywhere

Vocabulary

- **Class** - the blueprint
- **Object** - one thing built from the class
- **Data member** - variables inside the class
- **Member function** - functions inside the class

Object-Oriented Programming

Constructor runs automatically the moment an object is created. It is meant to initialize the data members and leave it in a usable state

Rules:

- Same name as the class
- No return type, not even void
- Called automatically when object is created
- Can have several just as long as parameters differ
- If there's no constructor then the compiler will supply an empty one

```
main(){  
    Student s1;  
    Student s2("Dara" , 18);  
}
```

```
class Student{  
private:  
    string name; // private  
    int age;     // private  
  
public:  
    Student() { // default constructor  
        name = "unknown";  
        age = 0;  
    }  
  
    Student(string n , int a) { // parameterized  
        name = n;  
        age = a;  
    }  
};
```


Object-Oriented Programming

Destructor runs automatically when an object is destroyed. If the object asked for heap memory, this is where it gives it back

Rules:

- Class name preceded by ~
- No return type, no parameters
- Called automatically when object is destroyed
- Exactly one per class
- Runs at end of scope or on delete

Why this matters:

With no destructor, every path out of a function needs its own `delete[]`. Miss one and you leak.

With a destructor, you write the `delete[]` once and it runs on every path, automatically, forever

```
class Course{
private:
    Student *students; // heap memory this object owns
    int count;

public:
    Course(int n){        // constructor asks
        students = new Student[n];
        count = n;
    }

    ~Course(){            // destructor gives back
        delete[] students;
        students = nullptr;
    }
};
```

Lab Exercise

1. Swap with Pointers

Write **void swap(int *a , int *b)** that swaps two values. Call it from **main** and print before and after.

2. Dynamic Array

Ask the user for a size. Allocate an **int** array of that size with **new**. Fill it, print it then release it properly.

3. Struct

Define **struct Book { string title; string author; int year; }**; Create an array of 3 books, fill them and print each one.

4. Struct Pointer

Point a **Book*** at your second book. Change its year using **->**, then print the array again to prove the original changed.

Lab Exercise

1. Convert

Turn your **Book struct** into a **class**. Make three members private and add **setTitle()**, **getTitle()** and your optional methods. Confirm that **b.title = "x"** no longer compiles.

2. Constructors

Add a **default constructor** and a **parameterized** one. Create one object each way and print both.

3. Class that owns memory

Write **class Library** holding a **Book *books**; on the heap plus **capacity** and a **count**. The constructor takes a capacity and allocates. Add **addBook()** and **printAll()**.

4. Destructor

Write **~Library()** to release the array. Put a **cout** in both the constructor and destructor then run and watch each message appears

References

- [1] Stroustrup, B. (2013). The C++ programming language (4th ed.). Addison-Wesley.
- [2] Deitel, P., & Deitel, H. (2017). C++ how to program (10th ed.). Pearson.
- [3] Memory management. (n.d.). cppreference.com.
<https://en.cppreference.com/w/cpp/memory>.
- [4] Dynamic memory allocation with new and delete. (n.d.). LearnCpp.com.
<https://www.learncpp.com/cpp-tutorial/dynamic-memory-allocation-with-new-and-delete/>

The slide features a light gray background with decorative vertical bars on the left and right sides. These bars are composed of several parallel, slightly offset rectangular strips in dark blue, green, and orange, creating a modern, geometric look. The text 'Thank You!' is centered in a large, orange, sans-serif font, with a thin orange horizontal line underneath it.

Thank You!